G-009:Weidong Zhu: Detecting Ransomware Attacks via Reverse Engineering the Ransomware Binaries

Tibo Gobet
Student ID: 6597336

Abstract

Ransomware has emerged as one of the most disruptive forms of malware, causing significant financial losses, operational disruptions, and security risks across industries worldwide. Traditional detection approaches, particularly signature-based antivirus systems, have become increasingly ineffective against modern ransomware due to the widespread use of obfuscation, packing, and frequent code modifications. This project addresses this limitation by proposing a lightweight and interpretable heuristic-based ransomware detection system using static binary analysis. The proposed system extracts multiple features from executable files, including Shannon entropy, Import Address Table (IAT) entries, embedded strings, and simplified behavioral indicators that approximate ransomware activity. These features are combined into a weighted scoring model that assigns a numerical risk score to each file. Experimental evaluation was conducted on a dataset of benign Windows executables and publicly available ransomware samples. Results show the system successfully distinguishes most benign files from suspicious ones, while partially identifying certain ransomware samples. Limitations emerge when analyzing obfuscated or minimal ransomware binaries that do not expose clear static indicators. This study demonstrates that heuristic-based static analysis can provide meaningful and interpretable detection capabilities, while highlighting the trade-offs between simplicity, accuracy, and robustness.

1 Introduction

Ransomware is a form of malware that encrypts user data and demands payment for its recovery, posing a significant threat to individuals, organizations, and critical infrastructure worldwide. As ransomware continues to evolve, traditional signature-based detection methods have become less effective, especially against new or modified variants that leverage obfuscation, packing, and frequent code changes to evade detection.

Alternative detection approaches such as dynamic behavioral analysis and machine learning have been widely explored. However, these methods often require significant computational resources, complex sandboxing environments, or lack transparency in their decision-making, making them difficult to interpret and deploy in resource-constrained settings.

This project proposes a lightweight, heuristic-based detection system using static binary analysis. Rather than relying on execution or complex models, the system extracts interpretable features—including Shannon entropy, API imports, and embedded strings—and combines them into a weighted scoring model to classify files as benign or potentially malicious. The objective is to evaluate whether simple, transparent heuristics can effectively distinguish between ransomware and benign software without the overhead of machine learning or dynamic analysis.

The remainder of this report is organized as follows: Section 2 provides background on ransomware and reverse engineering; Section 3 motivates the research; Section 4 describes the system design; Section 5 presents the evaluation; Section 6 reviews related work; and Section 7 concludes.

2 Background

Ransomware is among the most damaging types of malware currently affecting the cybersecurity landscape, both financially and operationally. Its impact spans industries, disrupting business continuity, government services, healthcare infrastructure, and critical systems. The widespread WannaCry attack demonstrated the global scale ransomware can reach, and as new variants continue to evolve, developing effective detection techniques remains an ongoing challenge.

The most common detection approach has traditionally been signature-based antivirus systems, which identify malicious files by comparing them against known malware patterns or hashes. However, modern ransomware routinely evades signature-based detection through obfuscation, packing, and minor code modifications. As noted by Clancy [5], malware authors deliberately design malicious code to be

difficult to analyze, frequently altering code structures to circumvent detection mechanisms—demonstrating the ineffectiveness of static signature matching alone against novel ransomware variants.

Malware reverse engineering addresses these limitations by analyzing executable binaries to understand their internal structure, design logic, runtime behavior, and functionality. Through reverse engineering, analysts can identify how malware operates, uncover encryption routines, trace command-and-control communication paths, and develop mitigation strategies.

Reverse engineering can be performed using static or dynamic analytical techniques. Static analysis examines the malware binary without executing it, allowing researchers to review disassembled code, imported APIs, entropy values, and embedded strings without risk to the host system [8]. Dynamic analysis executes the malware in a controlled sandbox environment to observe its behavior at runtime. While combining both approaches provides the most comprehensive view, static analysis offers the advantage of early-stage detection and operational safety.

Research institutions such as Carnegie Mellon University's Software Engineering Institute (SEI) have focused on automating reverse engineering workflows to improve speed and scalability. Tools such as Pharos demonstrate that automated binary analysis can significantly accelerate malware classification and similarity detection [4]. However, many of these automated systems are computationally intensive and lack transparency.

This project adopts a heuristic detection framework grounded in static binary analysis. Rather than relying on black-box machine learning models, the system assigns weighted scores to features that exhibit statistical correlation with ransomware behavior—such as high Shannon entropy indicative of packing or encryption, cryptographic API imports, file enumeration routines, shadow copy deletion commands, and other structural indicators of ransomware activity.

3 Motivation

The motivation for this project stems from a critical gap in current ransomware detection research. Signature-based detection systems fail against modern ransomware because they cannot generalize to new, obfuscated, or polymorphic variants. Dynamic analysis and machine learning approaches, while more capable, carry significant trade-offs: sandboxing environments are complex to set up and resource-intensive to operate, and are vulnerable to anti-analysis techniques employed by sophisticated malware. Machine learning models require large volumes of labeled training data and often function as black boxes, offering little interpretability into why a file was flagged [2, 3].

Existing research tends to fall into one of two extremes: highly automated, large-scale malware classification systems, or complex ML pipelines that sacrifice transparency for accuracy. There is a meaningful gap in lightweight, interpretable detection tools that can serve as a practical first line of defense without requiring extensive infrastructure.

This project addresses that gap by asking a fundamental question: *can a simple, rule-based heuristic scoring system—built on well-established static binary indicators—detect ransomware effectively without machine learning or dynamic execution?* The goal is not to replace more sophisticated techniques, but to determine the extent to which interpretable heuristics can provide actionable detection results, and to clearly document the conditions under which such an approach succeeds or fails.

4 Design

The system is designed as a static analysis pipeline that processes Windows Portable Executable (PE) files and assigns a heuristic risk score based on extracted binary features. A static-only approach is used to ensure both safety—since no potentially malicious code is executed—and efficiency, as analysis can be performed rapidly without a sandbox environment.

4.1 Feature Extraction

The system extracts the following features from each executable:

Shannon Entropy. Entropy is computed over the binary content of the file. Values above 7.0 are considered indicative of packing or encryption, which are common preprocessing steps in ransomware deployment.

Import Address Table (IAT) Analysis. The IAT is parsed to identify API calls associated with ransomware behavior. APIs are grouped into behavioral categories: file enumeration (e.g., `FindFirstFile`, `FindNextFile`), file manipulation (e.g., `ReadFile`, `WriteFile`), file deletion (e.g., `DeleteFile`), cryptographic operations (e.g., `CryptEncrypt`, `BCryptEncrypt`), and shadow copy deletion commands.

Embedded String Analysis. Strings extracted from the binary are scanned for ransomware-associated keywords such as "bitcoin," "ransom," ".onion," and "decrypt," commonly found in ransom notes or communication routines.

Simplified Behavioral Model. A lightweight behavioral approximation is constructed by counting the co-occurrence of API categories that match known ransomware workflow patterns—for example, the simultaneous presence of file enumeration and cryptographic APIs.

4.2 Scoring Model

Each extracted feature contributes a weighted score to a cumulative risk value. Weights were assigned based on the relative

discriminating power of each feature as documented in prior static analysis literature [6, 7, 9]. A threshold is applied to the final score to classify files as either *benign* or *suspicious*, selected empirically to minimize false positives on benign executables while maintaining sensitivity to known ransomware indicators. The design emphasizes interpretability: each classification decision can be traced back to specific features and their individual score contributions, providing transparency absent in black-box ML classifiers.

5 Evaluation

5.1 Experimental Setup

The system was evaluated on a dataset composed of benign Windows system executables and a set of publicly available ransomware samples. Each file was analyzed independently using the static analysis pipeline described in Section 4, producing a final heuristic score and a binary classification.

5.2 Results on Benign Files

Most benign files received low heuristic scores, confirming that the scoring model does not overly penalize normal software. For example, `calc.exe` produced a score of 0, indicating no suspicious features. However, certain legitimate programs such as `cmd.exe` and `notepad.exe` produced moderate scores due to the presence of file manipulation APIs in their IATs. These borderline cases highlight a known challenge in static analysis: normal system utilities can exhibit behavioral patterns superficially similar to ransomware, leading to potential false positives.

5.3 Results on Ransomware Samples

Detection results varied significantly across ransomware samples depending on the visibility of static features. Samples with high entropy values (above 7.0) were more reliably flagged, as elevated entropy contributed meaningfully to the cumulative risk score. However, several samples showed low entropy and limited IAT exposure, resulting in scores below the detection threshold. These samples were likely obfuscated or minimally structured, concealing their ransomware indicators from static inspection. String analysis also produced inconsistent results: some samples contained clear ransomware-related strings such as ransom note fragments or cryptocurrency addresses, while others contained none.

5.4 Discussion

The evaluation confirms that heuristic-based static analysis can successfully identify suspicious characteristics in ransomware samples that expose clear structural indicators. However, obfuscation and packing can render static features uninformative, effectively hiding the malicious nature of a binary until runtime. Ransomware detection is inherently a probabilistic problem: no single feature is sufficient for accurate classification, and even a combination of features may fail to capture all malicious behavior. These findings suggest that such a system is best deployed as a lightweight first-pass filter rather than a standalone solution.

6 Related Work

6.1 Signature-Based Detection

Early malware detection systems relied on signature matching, comparing executables against known malicious byte patterns or hashes. This approach is computationally efficient and widely deployed in commercial antivirus solutions. However, it does not generalize to polymorphic or obfuscated ransomware and provides no protection against zero-day attacks, as it can only detect previously catalogued malware families.

6.2 Dynamic and Behavioral Analysis

Dynamic analysis systems run malware in secure sandbox environments and monitor behavioral indicators such as rapid file encryption loops, shadow copy deletion (`vssadmin`), abnormal file I/O patterns, and registry modifications. While effective at exposing runtime behavior, dynamic analysis requires significant infrastructure, is vulnerable to sandbox-detection evasion, and incurs higher computational costs.

6.3 Static Feature-Based and ML Approaches

Post-2020 research has increasingly examined static binary properties for malware classification. Common features include Shannon entropy, IAT analysis, opcode frequency distributions, section attributes, and embedded strings. Machine learning models trained on these features have demonstrated strong detection accuracy [2, 3, 6, 7, 9]. However, ML-based approaches require large labeled datasets, introduce model opacity, and carry higher implementation complexity. The classification of ransomware families using N-gram opcode features has also shown promise for capturing behavioral patterns at the binary level [1].

6.4 Automated Reverse Engineering Systems

Institutions such as Carnegie Mellon University's SEI have developed automated reverse engineering frameworks to improve malware classification and similarity detection at scale [4]. While these systems greatly reduce manual analysis effort, they are not designed specifically for lightweight heuristic detection of ransomware.

6.5 Research Gap

Prior work largely relies on runtime behavioral analysis, computationally intensive ML pipelines, or large-scale automated classification. There is limited research on interpretable, lightweight heuristic scoring systems combining static structural features with simplified behavioral indicators derived from reverse engineering—the gap this project addresses.

7 Conclusion

This project presented a heuristic-based ransomware detection system using static binary analysis. By combining Shannon entropy, IAT-based API inspection, embedded string analysis, and simplified behavioral pattern matching into a weighted scoring model, the system classifies executable files as benign or potentially malicious without executing any code.

The evaluation demonstrates that interpretable heuristics provide meaningful detection capability for ransomware samples exposing clear static indicators. At the same time, obfuscated or minimally structured samples evade detection, and certain benign utilities produce borderline scores—underscoring the difficulty of distinguishing legitimate from malicious file operations using static features alone.

Future work should explore integrating dynamic analysis to capture runtime behavior invisible to static inspection, refining feature selection to reduce false positives, and combining heuristic methods with machine learning in a hybrid pipeline that preserves interpretability while improving robustness against obfuscated variants.

Ethical Considerations

This project involves the analysis of publicly available ransomware samples for research purposes only. No malicious code was executed on live systems; all analysis was performed statically on binary files in an isolated environment. Ransomware samples were obtained from publicly accessible malware repositories commonly used in academic cybersecurity research. No new malware was created or distributed as part of this work. The goal of this research is strictly defensive: to improve detection capabilities and contribute to protecting individuals and organizations from ransomware.

Open Science

The artifacts for this project include the source code implementing the static analysis and heuristic scoring pipeline, the list of benign executables used for evaluation, references to the public malware repositories from which ransomware samples were obtained, and a `README.md` describing how to run the tool. These artifacts are submitted as a `.zip` file alongside this report. Ransomware samples are not redistributed directly; instead, references to their original public sources are provided to allow reproduction of the evaluation environment responsibly.

References

[1] Classification of ransomware families with machine learning based on N-gram of opcodes. *Future Generation Computer Systems*, 90:211–221, 2019. https://doi.org/10.1016/j.future.2018.07.054.

[2] Malak Aljabri, Fahd Alhaidari, Aminah Albuainain, Samiyah Alrashidi, Jana Alansari, Wasmiyah Alqahtani, and Jana Alshaya. Ransomware detection based on machine learning using memory features. *Egyptian Informatics Journal*, 25:100445, 2024. https://doi.org/10.1016/j.eij.2024.100445.

[3] Iman Almomani, Aala AlKhayer, and Walid El-Shafai. E2E-RDS: Efficient end-to-end ransomware detection system based on static-based ML and vision-based DL approaches. *Sensors*, 23(9):4467, 2023. https://doi.org/10.3390/s23094467.

[4] Carnegie Mellon University Software Engineering Institute. Reverse engineering for malware analysis. Technical report, April 2024. https://insights.sei.cmu.edu/blog/reverse-engineering-for-malware-analysis/.

[5] R. Clancy. A quick guide to reverse engineering malware, October 2022. Cybersecurity Exchange. https://www.eccouncil.org/cybersecurity-exchange/ethical-hacking/reverse-engineering-malware-beginners/.

[6] Kamdan, Y. Pratama, R. S. Munzi, A. B. Mustafa, and I. L. Kharisma. Static malware detection and classification using machine learning: A random forest approach. In *Proceedings of the 7th International Global Conference Series on ICT Integration in Technical Education & Smart Society*, volume 76, 2025. https://doi.org/10.1145/3721931.3721943.

[7] K. Lee, J. Lee, S.-Y. Lee, and K. Yim. Effective ransomware detection using entropy estimation of files for cloud services. *Sensors*, 23(6):3023, 2023. https://doi.org/10.3390/s23063023.

[8] Sudip Sengupta. Reverse engineering malware: Techniques and tools for analyzing and dissecting malicious software, April 2023. Medium. https://medium.com/@sudipsengupta.

[9] A. Venčkauskas, V. Jusas, and D. Barisas. Zero-day ransomware attack detection using static portable executable header features. *Applied Sciences*, 15(9):10576, 2025. https://doi.org/10.3390/app15095010576.